entropy

MDPI

Article

Transition-Aware Decomposition of Single-Qudit Gates

Denis A. Drozhzhin, Evgeniy O. Kiktenko, Aleksey K. Fedorov and Anastasiia S. Nikolaeva *

Laboratory of Quantum Information Technologies, National University of Science and Technology "MISIS", Moscow 119049, Russia; denisdrozhzhin1999@gmail.com (D.A.D.)
* Correspondence: anastasiia.nikolaeva21@gmail.com

Abstract

Quantum computation with d-level quantum systems, also known as qudits, benefits from the possibility to use a richer computational space compared to qubits. However, for an arbitrary qudit-based hardware platform, the issue is that a generic qudit operation has to be decomposed into the sequence of native operations—pulses that are adjusted to the transitions between two levels in a qudit. Typically, not all levels in a qudit are simply connected to each other due to specific selection rules. Moreover, the number of pulses plays a significant role, since each pulse takes a certain execution time and may introduce error. In this paper, we propose a resource-efficient algorithm to decompose single-qudit operations into the sequence of pulses that are allowed by qudit selection rules. Using the developed algorithm, the number of pulses is at most $d(d-1)/2$ for an arbitrary single-qudit operation. For specific operations, the algorithm could produce even fewer pulses. We provide a comparison of qudit decompositions for several types of trapped ions, specifically $^{171}Yb^+$, $^{137}Ba^+$ and $^{40}Ca^+$ with different selection rules, and also decomposition for superconducting qudits. Although our approach deals with single-qudit operations, the proposed approach is important for realizing two-qudit operations since they can be implemented as a standard two-qubit gate that is surrounded by efficiently implemented single-qudit gates.

Keywords: qudits; quantum circuits; quantum algorithms; single-qudit gates; trapped ions; superconductors; quantum computing

1. Introduction

The development of a large-scale quantum computer [1–3] that is able to outperform devices based on classical principles in practically relevant problems, such as simulating complex (quantum) systems [4], combinatorial optimization [5], or prime factorization [6], represents an outstanding challenge. Various computational models, for example, digital quantum computing [1,2], variational quantum algorithms [7,8], or programmable quantum simulation [9], and diverse physical platforms, such as superconducting circuits [10,11], semiconductor quantum dots [12–14], optical systems [15,16], neutral atoms [9,17–19], and trapped ions [20–22], are under study on the way to useful quantum computers. The inherent idea is to find suitable conditions for scaling quantum devices with respect to the number of available information carriers without significantly degrading the quality of the control. In the most well-developed digital model of quantum computing, information carriers are qubits [1–3], which are two-level quantum counterparts of classical bits. The execution of quantum algorithms requires the realization of single- and two-qubit operations under a register of qubits, so that the combination of the number of qubits and quality of quantum operations (gates) is the crucial parameter.

Check for updates

Academic Editor: Xiao Yuan

Received: 8 November 2025
Revised: 17 December 2025
Accepted: 27 December 2025
Published: 31 December 2025

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Underlying physical platforms, such as trapped ions or neutral atoms, however, almost always have Hilbert spaces of higher dimensionalities, which gives interesting possibilities [23]. Computing models based on the use of additional degrees of freedom of physical systems, which makes them qudits (where d indicates that the dimension of the Hilbert space may be larger than two), attract great interest since they allow scalability of quantum computing devices without the need for an increase in the number of physical carriers [24–61]. There are several approaches to how qudits can be used for more efficient quantum processors. First, one can think about a multilevel system with 2^n levels as a set of qubits: for example, a ququart is equivalent to a two-qubit system [62–64]. At the same time, implementing a two-qubit operation with a single ququart (strictly speaking, this is a single-qudit operation) would not require physical interaction between physical objects, so the fidelity of such operations can be as high as the fidelity of single-qubit operations. The second approach is the use of additional qudit levels to substitute ancilla qubits in multiqubit gate decompositions [65] (for example, for the Toffoli gate [36,38,40,43,66–71]). However, the efficiency of these methods and their combination depends on the mapping, i.e., the way in which qubits are encoded in qudits [62,64].

As in the case of conventional qubit-based processors, the choice of a specific physical platform is also important for qudit setups. Recently, multiqudit quantum processors [72–75], including systems based on superconducting transmon qudits [72] and photons [73] as well as $^{40}Ca^+$ [74] and $^{171}Yb^+$ [75] ion qudits, have been demonstrated. It is clear that in the case of atomic qubits (ions or neutral atoms), qudit encoding is natural due to multilevel structures, and it is straightforward to address more than two levels using a single laser with acousto-optic modulators (AOMs) [74,75]. Superconducting [76,77] and photonic [78] systems are also promising for qudit-based quantum computing. Recent progress in quantum computing with qudits also includes the demonstration of a significant improvement in the realization of multiqubit gates [79,80] and quantum algorithms with qudits [39,76]. However, various details of resource-efficient implementation of quantum algorithms with qudits are the subject of research.

In this work, we address a specific problem of the realization of single-qudit operations, taking into account selection rules of physical systems. The objective is to minimize the number of pulses in order to realize a single-qudit operation, since such a minimization would result in shorter execution time and a lower number of errors. We propose a resource-efficient algorithm to decompose qudit operations into the sequence of pulses that are allowed by qudit selection rules. As we demonstrate, in the proposed algorithm, the number of pulses is at most $d(d-1)/2$ for an arbitrary single-qudit operation, or even fewer pulses for specific single-qudit operations. We provide a comparison of qudit decompositions for several types of trapped ions, specifically $^{171}Yb^+$, $^{137}Ba^+$ and $^{40}Ca^+$ with different selection rules, and also decomposition for superconducting qudits.

2. Qudit Computing

A common quantum device relies on operations with *qubits*, i.e., the set of transformations of two-level systems in the superposition states:

$$|\psi\rangle_{\mathbf{qb}} = \psi_0 |0\rangle + \psi_1 |1\rangle . \tag{1}$$

In the case of *qudits*, the structure of the superposition state of the d-level quantum system is more complex:

$$|\psi\rangle_{\mathbf{qd}} = \psi_0 |0\rangle + \cdots + \psi_{d-1} |d-1\rangle = \sum_{n=0}^{d-1} \psi_n |n\rangle , \tag{2}$$

where d is the dimension of the quantum state.

The evolution of a qudit state can be described using unitary operators, similarly to the case of qubit operators. For example, if we interpret qubit Pauli σ_x as an operation that swaps two levels, then we obtain:

$$\mathsf{SWAP}^{ij}_{\mathbf{qd}} = |i\rangle\langle j| + |j\rangle\langle i| + \sum_{n\neq i,j} |n\rangle\langle n| \,. \tag{3}$$

On the other hand, σ_x can be generalized to modulo d increment:

$$\mathsf{X}_{\mathbf{qd}} = \sum_n |n{+}1 \bmod d\rangle\langle n| \,, \tag{4}$$

$$\mathsf{X}^k_{\mathbf{qd}} = \sum_n |n{+}k \bmod d\rangle\langle n| \,. \tag{5}$$

Also, one can use phase gates $\mathsf{Z}_{\mathbf{qd}}$ and the d-nary Hadamard transform $\mathsf{H}_{\mathbf{qd}}$, which is equivalent to the Quantum Fourier Transform on a single qudit:

$$\mathsf{Z}_{\mathbf{qd}} = \sum_n \omega^n |n\rangle\langle n| \,, \tag{6}$$

$$\mathsf{H}_{\mathbf{qd}} = \sum_{n,m} \frac{\omega^{nm}}{\sqrt{d}} |n\rangle\langle m| \,, \tag{7}$$

where $\omega = e^{\imath 2\pi/d}$ is the dth root of unity.

In general, any single-qudit evolution can be described using elements of the unitary group $\mathsf{U}_{\mathbf{qd}} \in U(d)$. This group is isomorphic to the group of $d{\times}d$ matrices with complex entries that satisfy the unitary condition:

$$\mathsf{U}_{\mathbf{qd}} \equiv \begin{Vmatrix} \mathcal{U}_{0,0} & \mathcal{U}_{0,1} & \cdots & \mathcal{U}_{0,d-1} \\ \mathcal{U}_{1,0} & \mathcal{U}_{1,1} & \cdots & \mathcal{U}_{1,d-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{U}_{d-1,0} & \mathcal{U}_{d-1,1} & \cdots & \mathcal{U}_{d-1,d-1} \end{Vmatrix} \tag{8}$$

$$\mathsf{U}_{\mathbf{qd}}{}^{\dagger}\mathsf{U}_{\mathbf{qd}} = \mathsf{Id} \equiv \sum_{k=0}^{d-1} \mathcal{U}^*_{k,n}\,\mathcal{U}_{k,m} = \delta_{n,m} \tag{9}$$

The knowledge of how to efficiently implement arbitrary single-qudit evolution with existing qudit hardware is the crucial step towards the universal qudit computer, especially for systems with high dimensionality [76,77,81–83].

Another important part of computation with qudits is entangling operations. Using the extension of Pauli operations to qudits, we obtain the analogues of qubit controlled gates, particularly CX and CZ:

$$\mathsf{CU}_{\mathbf{qd}} = \sum_{m=0}^{d-1} |m\rangle\langle m| \otimes \mathsf{U}^m_{\mathbf{qd}}, \tag{10}$$

$$\mathsf{CX}_{\mathbf{qd}} = \sum_{m,n} |m, n{+}m\rangle\langle m, n| \,, \tag{11}$$

$$\mathsf{CZ}_{\mathbf{qd}} = \sum_{m,n} \omega^{mn} |m, n\rangle\langle m, n| \,. \tag{12}$$

Also, on several trapped-ion platforms, the main two-qudit operation is Mølmer-Sørensen gate [74,84]:

$$\mathsf{MS}(\chi) = \exp\left\{-\imath\frac{\chi}{2}\left(\sigma_x^{01}\otimes 1 + 1\otimes\sigma_x^{01}\right)^2\right\}, \tag{13}$$

which is equivalent up to global phase to qubit RXX operation on the subspace $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$:

$$\mathsf{RXX}(\chi) = \exp\{-\imath\chi\, \sigma_x \otimes \sigma_x\}. \tag{14}$$

Although we do not consider qudit entangling operations within the scope of this paper, we are able to embed n-qubit gates into a single-qudit operation if $d \geq 2^n$.

3. Qudit Hardware Operations

From an experimental point of view, the qudit system can be implemented using different physical systems: trapped ions, neutral atoms, superconducting circuits, photonic circuits, etc.

A qudit state can be encoded into ions' or neutral atoms' degrees of freedom, e.g., states of the electron on the outer shell [82–84]. In this model, each state $|n\rangle$ has the corresponding energy E_n. After applying the laser field with a frequency $\frac{1}{\hbar}|E_i - E_j|$, the Rabi oscillation of the electron between states $|i\rangle$ and $|j\rangle$ is stimulated if this transition is permitted by selection rules.

Superconducting circuits are able to store qudit states as Fock states of Copper pairs in the potential well [76,77]. Transitions between states are also stimulated via Rabi pulses between neighboring states $|n\rangle$ and $|n{+}1\rangle$. The potential well should provide some degree of anharmonicity so that each transition can be addressed individually.

Photonic circuits can represent a qudit state using d-wire schemes with a single photon [73,78]. An index of the basis qudit state represents the index of the wire. Using the beam-splitter with a given amplitude reflection rate r (it is usually set $r{=}1/\sqrt{2}$ for a half-transparent mirror), we can permute photons in two neighboring wires. Also, we can perform a phase-shift of a single wire at an arbitrary angle.

We can note that every mentioned qudit platform supports two main types of operations. The first is the phase shift on the level $|k\rangle$, which we denote $\mathsf{P}^k(\theta)$. The second is a two-level transition $\mathsf{R}^{ij}(\theta,\phi)$ between levels $|i\rangle$ and $|j\rangle$. These operations have the following definitions:

$$\mathsf{P}^k(\theta) = \exp\left\{\imath\theta\, |k\rangle\langle k|\right\}, \tag{15}$$

$$\mathsf{R}^{ij}(\theta,\phi) = \exp\left\{-\imath\frac{\theta}{2}\sigma_\phi^{ij}\right\}, \tag{16}$$

where $\sigma_\bullet^{ij}$ is the qudit analogue of qubit Pauli operators acting on the two-level subspace $\{|i\rangle, |j\rangle\}$ of a qudit:

$$\sigma_x^{ij} = |j\rangle\langle i| + |i\rangle\langle j|, \tag{17}$$

$$\sigma_y^{ij} = \imath\,|j\rangle\langle i| - \imath\,|i\rangle\langle j|, \tag{18}$$

$$\sigma_\phi^{ij} = e^{+\imath\phi}\,|j\rangle\langle i| + e^{-\imath\phi}\,|i\rangle\langle j|. \tag{19}$$

Thus, R^{ij} operation can be viewed as a 2×2 unitary matrix R that acts on subspace spanned by $\{|i\rangle, |j\rangle\}$:

$$\mathsf{R}(\theta,\phi) = \begin{Vmatrix} \cos\left(\frac{\theta}{2}\right) & -\imath e^{-\imath\phi}\sin\left(\frac{\theta}{2}\right) \\ -\imath e^{+\imath\phi}\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{Vmatrix}. \tag{20}$$

Several platforms may lack two-parametric transition operations, e.g., for photonic circuits, there is only a single parameter r in the beam-splitter operation:

$$\mathsf{T}(r) = \begin{Vmatrix} r & -\sqrt{1-r^2} \\ \sqrt{1-r^2} & \sqrt{r} \end{Vmatrix}. \tag{21}$$

We may always apply phase shifts and transform this operation into an R^{ij} operation:

$$\mathsf{R}^{ij}(\theta,\phi) = \mathsf{P}^j\left(\phi - \frac{\pi}{2}\right) \circ \mathsf{T}^{ij}\left(\cos\frac{\theta}{2}\right) \circ \mathsf{P}^j\left(\frac{\pi}{2} - \phi\right). \tag{22}$$

We also consider arbitrary selection rules for a given qudit physical implementation that permit applicable R^{ij} operations. In the following sections, we demonstrate that any qudit operation U can be decomposed into the sequence of allowed R^{ij} and P^k operations. Moreover, for any selection rules, we guarantee at most $d(d-1)/2$ transition operations R^{ij} in the final decomposition.

4. Unitary Decomposition for Trapped-Ion and Superconducting Hardware

Existing decomposition schemes utilize superconducting selection rules: transitions are allowed only for neighboring states. Two distinct schemes are provided in [85] (*Row-by-row* scheme for superconducting circuits) and in [86] (*Square* scheme for photonic circuits). Both schemes perform unitary decomposition by eliminating non-diagonal entries from the unitary matrix U using a sequence of two-level R operations.

In these decompositions, we initialize the algorithm with a unitary $d \times d$ matrix. After right-applying a two-level operation $\mathsf{R}^{01}(-\theta_1, \phi_1)$ to U, elements of the first two columns are modified in the following way:

$$\begin{Vmatrix} \mathcal{U}'_{k,0} \\ \mathcal{U}'_{k,1} \end{Vmatrix} = \mathsf{R}(-\theta_1, \phi_1)^T \begin{Vmatrix} \mathcal{U}_{k,0} \\ \mathcal{U}_{k,1} \end{Vmatrix}. \tag{23}$$

By choosing proper values for parameters θ_1 and ϕ_1, we can set a single element of the first column $\mathcal{U}'_{d-1,0}$ to 0:

$$\cos\left(\frac{\theta_1}{2}\right)\mathcal{U}_{d-1,0} + \imath e^{\imath\phi_1} \sin\left(\frac{\theta_1}{2}\right)\mathcal{U}_{d-1,1} = 0, \tag{24}$$

$$\theta_1 = 2\arctan\left(\frac{|\mathcal{U}_{d-1,0}|}{|\mathcal{U}_{d-1,1}|}\right), \tag{25}$$

$$\phi_1 = \frac{\pi}{2} + \arg(\mathcal{U}_{d-1,0}) - \arg(\mathcal{U}_{d-1,1}). \tag{26}$$

In the next steps, we apply operations $\mathsf{R}^{n-1,n}$ in order to eliminate the element $\mathcal{U}_{d-1,n-1}$ using element $\mathcal{U}_{d-1,n}$. After $d-1$ steps, all elements in a row $d-1$ are eliminated. Due to the unitary property, all elements in a column $d-1$ are also eliminated, and the diagonal element has absolute value 1. With the phase gate $\mathsf{P}^{d-1}(\alpha_{d-1} = \arg(\mathcal{U}_{d-1,d-1}))$, we obtain the following decomposition:

$$\begin{aligned} \mathsf{U}_d = & \begin{Vmatrix} \mathsf{U}_{d-1} & 0 \\ 0 & 1 \end{Vmatrix} \circ \\ & \circ\, \mathsf{P}^{d-1}(\alpha_{d-1}) \circ \mathsf{R}^{d-2,d-1}(\theta_{d-1}, \phi_{d-1}) \circ \\ & \circ\, \cdots \circ \\ & \circ\, \mathsf{R}^{1,2}(\theta_2, \phi_2) \circ \mathsf{R}^{0,1}(\theta_1, \phi_1). \end{aligned} \tag{27}$$

After eliminating the row, we can repeat the same algorithm on the lesser $(d-1)\times(d-1)$ unitary matrix U_{d-1}. The full row elimination scheme for a superconducting qudit is depicted in Figure 1a. Overall, this scheme requires at most $d(d-1)/2$ two-level transitions.

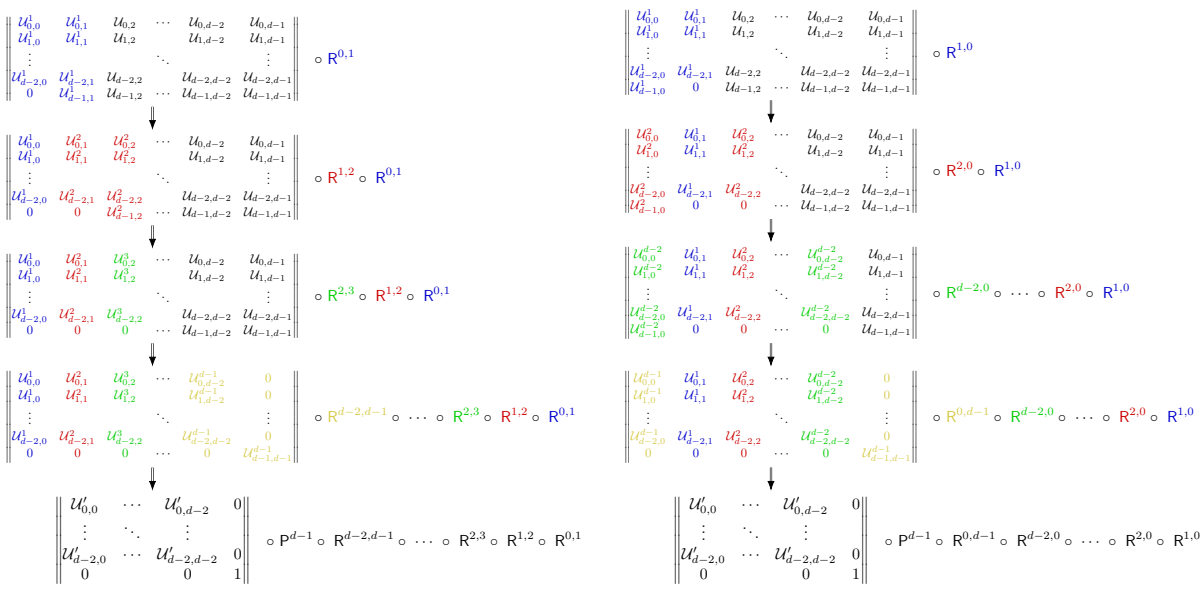


Figure 1. Row elimination schemes for a qudit unitary matrix with given allowed transitions. (**a**) Decomposition for a superconducting qudit with allowed transitions $\mathsf{R}^{n,n\pm1}$. (**b**) Decomposition for a trapped-ion qudit with allowed transitions $\mathsf{R}^{0,n}$ and $\mathsf{R}^{n,0}$.

If we want to implement the similar algorithm on the trapped-ion quantum computer [84] with $d=4$ levels, we face the following issue. Consider the device operating with transitions 01, 02, 03, whereas the given decomposition scheme operates with 01, 12, 23. The naive approach requires using level-swap operations that transform any transition R^{ij} into the allowed transition R^{0i}:

$$\begin{gathered}\mathsf{R}^{ij}(\theta,\phi)=\mathsf{S}^{0i}\circ\mathsf{R}^{0j}(\theta,\phi)\circ\mathsf{S}^{i0},\\ \text{or}\\ \mathsf{R}^{ij}(\theta,\phi)=\mathsf{S}^{0j}\circ\mathsf{R}^{i0}(\theta,\phi)\circ\mathsf{S}^{j0},\end{gathered}\tag{28}$$

$$\mathsf{S}^{ij}=\mathsf{R}^{ij}\left(\pi,\frac{\pi}{2}\right)\equiv\begin{Vmatrix}0 & -1\\ 1 & 0\end{Vmatrix}.\tag{29}$$

However, using this strategy for arbitrary unitary, we obtain 12 trapped-ion native transitions, instead of 6 as in the case of superconducting qudit. The best we can get using level-swaps is 8 native transitions. This number will grow more rapidly than the estimation $\leq d(d-1)/2$ for a superconducting qudit.

Hence, we should deduce the most optimal decomposition scheme for trapped-ion qudits, which typically have a star-like level transition graph. In general, we consider d levels trapped-ion qudit with transitions $\mathsf{R}^{0,n}$ or $\mathsf{R}^{n,0}$, where $0<n<d$. The main idea of our algorithm is to eliminate all row elements $\mathcal{U}_{d-1,n}$ where $0<n<d$ using the zeroeth element $\mathcal{U}_{d-1,0}$. Then, apply $\mathsf{R}^{0,d-1}$ to eliminate the zeroeth element as well. The full row elimination scheme for a trapped-ion qudit unitary is depicted in Figure 1b.

Row elimination for $d\times d$ unitary requires at most $(d-1)\times$ R and $1\times$ P gates. In total, after decomposing all lesser matrices, we obtain $d(d-1)/2\times$ R and $d\times$ P gates in unitary decomposition. Though some θ_n parameters could be equal 0 (if eliminated elements are

already $= 0$), hence these operations can be omitted in the hardware execution. Also, on both considered platforms, phase gates P can be performed virtually, so their amount in the decomposition does not affect execution time as much as transition gates R.

5. Unitary Decomposition for Qudit with Arbitrary Selection Rules

Our goal is to generalize the considered decomposition schemes to the case of arbitrary selection rules. A specific decomposition pattern could be spotted in the earlier proposed schemes. We apply $d-1$ operations in order to eliminate all non-diagonal terms. Each R operation requires an additional non-zero row element to perform elimination. This pattern to eliminate a row r could be described in a more formal way:

- On each step, we apply $\mathsf{R}^{z_i p_i}$ gate, where z_i is the index of eliminated element in row and p_i is the index of pivot element that used to eliminate z_i;
- z_i, p_i must be allowed transition;
- Indices z_i are non-repetitive and take all levels $\{0, 1, \ldots, r-1\}$;
- Indices p_i could be arbitrary levels, except those that have already been eliminated:

$$\forall i < r, j < i \Rightarrow p_i \neq z_j. \tag{30}$$

In fact, we are not restricted to eliminating only the last row in the unitary matrix. For a given sequence for eliminated rows $\{r_n\}_{n=0}^{d-1}$ (elimination from top r_{d-1} to bottom r_0), we just have to modify the 3rd rule in the pattern. On k^{th} step, where row r_k is eliminated, we should run through indices z_i that are non-repetitive and take all levels $\{r_0, r_1, \ldots, r_{k-1}\}$;

In total, the decomposition scheme consists of indices:

- r_k—the order of eliminated rows;
- z_{ki}—the order of eliminated elements on k^{th} step;
- p_{ki}—pivot elements on k^{th} step.

Using defined rules, we could produce a decomposition scheme for all arbitrary selection rules. The existence of such decomposition is not obvious yet but will be proved during the rest of the paper. However, indices for some selection rules, e.g., for a superconducting qudit and trapped-ion qudit, are already obtained in the previous section and depicted in Table 1.

We note that these rules are not sufficient to get a unique decomposition scheme. For example, in the trapped-ion case, we can eliminate the first $k-1$ elements in any order and still get the row eliminated. Also, a trapped-ion qudit with 5 levels can utilize two different schemes with different row orders. This degree of freedom can be useful, since we are allowed to select a preferable sequence of operations according to physical heuristics of levels and transitions of the given qudit platform.

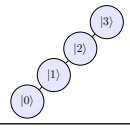
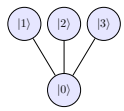
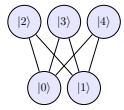
An arbitrary single-qudit system could be described as a connected undirected graph $\mathcal{G}$, where nodes $\mathcal{N}$ represent qudit levels and edges $\mathcal{E}$ represent allowed transitions. To determine decomposition indices for a given graph $\mathcal{G}$, we propose the following algorithm:

- On the step k (runs from $d-1$ to 1), $\mathcal{N}$ has $k+1$ nodes/levels. We select levels that, if removed, do not break connectivity of the graph. One of these levels can be chosen as $|r_k\rangle$ to eliminate the row r_k of the qudit unitary matrix (Figure 2a). Here, we can use physical heuristics to eliminate less stable levels earlier.
- Each level $|i\rangle \in \mathcal{G}$ is assigned the shortest distance from the level $|r_k\rangle$. This could be done using breadth-first search, which has time complexity $O(|\mathcal{N}| + |\mathcal{E}|)$. We can split all levels into sets $\{F_l\}$, where F_l contains levels at distance l from $|r_k\rangle$ (Figure 2b).
- Let L be the maximum distance from $|r_k\rangle$ in the $\mathcal{G}$. Each level $|z_{L,i}\rangle \in F_L$ is connected by a transition to some element $|p_{L,i}\rangle \in F_{L-1}$. We add these indices into the decomposition scheme, then repeat for all F_l in descending order of l (Figure 2c). There could be several levels $\in F_{L-1}$ connected to $|z_{L,i}\rangle$ (Figure 2d). Using physical properties

of a qudit, we can select the least noisy transition to reduce the average error of hardware execution.

- Continue the algorithm on the step $k-1$ for the transition graph $\mathcal{G}' = \mathcal{G}/\{|r_k\rangle\}$ until $k = 0$.

Table 1. Decomposition scheme indices for several qudit platforms: superconducting with 4 levels and trapped-ion with 4 and 5 levels.

Platform	d	Connectivity	Decomposition Schemes		
Superconducting qudit	4		$r_3 = 3$	$z_3 = \{0,1,2\}$	$p_3 = \{1,2,3\}$
			$r_2 = 2$	$z_2 = \{0,1\}$	$p_2 = \{1,2\}$
			$r_1 = 1$	$z_1 = \{0\}$	$p_1 = \{1\}$
$^{171}Yb^+$ trapped-ion qudit	4		$r_3 = 3$	$z_3 = \{1,2,0\}$	$p_3 = \{0,0,3\}$
			$r_2 = 2$	$z_2 = \{1,0\}$	$p_2 = \{0,2\}$
			$r_1 = 1$	$z_1 = \{0\}$	$p_1 = \{1\}$
$^{171}Yb^+$ trapped-ion qudit	5		$r_4 = 1$	$z_4 = \{0,4,3,2\}$	$p_4 = \{2,1,1,1\}$
			$r_3 = 4$	$z_3 = \{3,2,0\}$	$p_3 = \{0,0,4\}$
			$r_2 = 3$	$z_2 = \{2,0\}$	$p_2 = \{0,3\}$
			$r_1 = 2$	$z_1 = \{0\}$	$p_1 = \{2\}$
			$r_4 = 4$	$z_4 = \{3,2,1,0\}$	$p_4 = \{0,0,4,4\}$
			$r_3 = 3$	$z_3 = \{2,1,0\}$	$p_3 = \{0,3,3\}$
			$r_2 = 1$	$z_2 = \{0,2\}$	$p_2 = \{2,1\}$
			$r_1 = 2$	$z_1 = \{0\}$	$p_1 = \{2\}$

Produced decomposition indices satisfy rules required for correct row elimination:

- Indices $z_{l,i}$ are taken from each $\{F_l\}_{l=1}^{L}$. These are non-intersecting sets, so $\{|z_i\rangle\} = \mathcal{G}/\{|r_k\rangle\}$, i.e., all elements are eliminated except the diagonal one.
- Indices $p_{l,i}$ do not intersect already eliminated elements, since they have distance $> l$.

We are able to estimate the overall time complexity of proposed algorithm. Each step has complexity $O(|\mathcal{N}| + |\mathcal{E}|)$ from the breadth-first search and $O(|\mathcal{N}|)$ from assigning physical rotations. Thus, they are summed up to $O(d^2 + Td)$ complexity, where T is the total number of allowed transitions within the qudit.

Note that on each step transition graph remains connected; this property is crucial for correct unitary decomposition. If we have a graph with two or more disconnected components, then we can only obtain a non-zero element per component. Each of these non-zero elements could not be eliminated, since they are not connected via any R path.

The similar graph algorithm for the transition graph of the ^{86}Rb atom qudit was mentioned in [81]. Though it describes the general technique, we propose an algorithm that is able to minimize physical execution error and reduce the number of transitions for sparse unitary matrices.

The *static* decomposition scheme is obtained once for a given qudit platform. Then, this scheme can be used for any qudit unitary matrix that should be executed on a given qudit. However, for some sparse matrices with zero non-diagonal elements, better-optimized decomposition may be performed using the *adaptive* decomposition scheme (Figure 3). The adaptive decomposition runs through the same step as static decomposition for each unitary matrix. The difference is that a level is excluded from the graph during decomposition if it corresponds to a zero element and does not break graph connectivity. Also, the order of eliminated rows is determined by the matrix sparsity pattern: rows with fewer non-zero

elements are eliminated earlier. To sum up, the *adaptive* decomposition scheme is only valid for a given matrix and contains the least number of operations, whereas the *static* decomposition scheme can be applied to an arbitrary matrix.

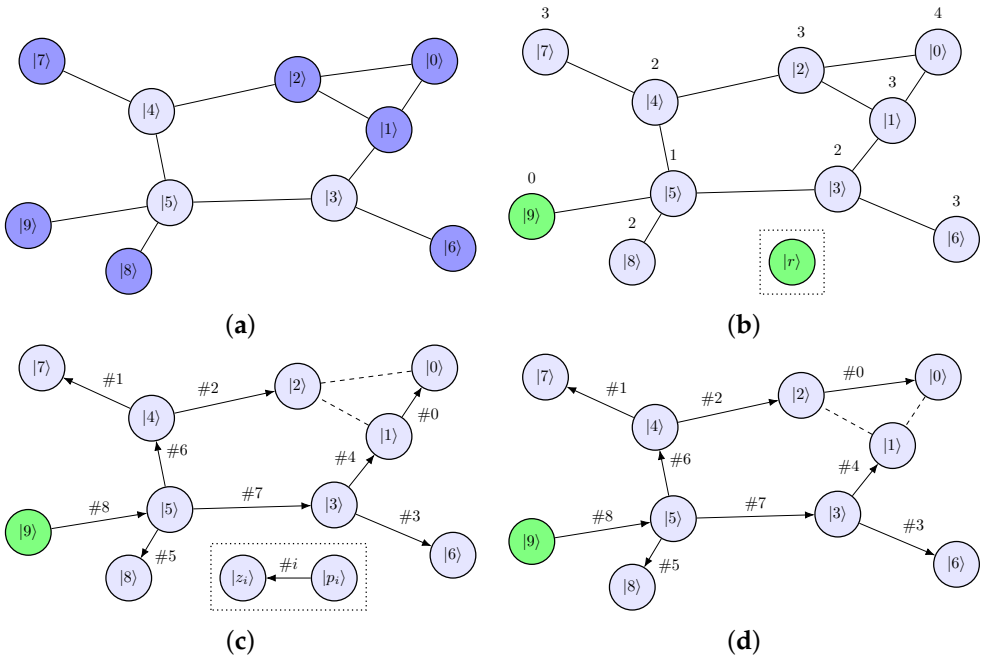


Figure 2. Algorithm that determines indices for the qudit decomposition scheme. It utilizes the graph structure of selection rules and the breadth-first search algorithm. (**a**) Select $|r\rangle$ candidates that can be removed without breaking connectivity of the graph. (**b**) Split the transition graph into sets $\{F_l\}$ using breadth-first search. $\forall l, |n\rangle \in F_l \rightarrow \mathrm{dist}(|r\rangle, |n\rangle) = l$. (**c**) Add indices $|z_i\rangle$ and $|p_i\rangle$ into the decomposition scheme for row $|r\rangle$. (**d**) Degree of freedom is presented in this algorithm. Qudit physical properties could be used to choose the best transition sequence.

$$\begin{Vmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ \mathcal{U}_{3,0} & \mathcal{U}_{3,1} & 0 & \mathcal{U}_{3,3} \end{Vmatrix} = \begin{Vmatrix} * & * & * & 0 \\ * & * & * & 0 \\ * & * & * & 0 \\ 0 & 0 & 0 & 1 \end{Vmatrix} \circ \mathsf{P}^3 \circ \mathsf{R}^{23} \circ \mathsf{R}^{13} \circ \mathsf{R}^{02} \qquad \begin{Vmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ \mathcal{U}_{3,0} & \mathcal{U}_{3,1} & 0 & \mathcal{U}_{3,3} \end{Vmatrix} = \begin{Vmatrix} * & * & * & 0 \\ * & * & * & 0 \\ * & * & * & 0 \\ 0 & 0 & 0 & 1 \end{Vmatrix} \circ \mathsf{P}^3 \circ \mathsf{R}^{13} \circ \mathsf{R}^{01}$$

Figure 3. Decomposition of a matrix with zero non-diagonal elements: (**a**) *Static* and (**b**) *Adaptive* schemes.

6. Comparison with Existing Decomposition Methods

To show the advantage of the developed algorithm, we provide the comparison with other approaches that provide the functionality of decomposing arbitrary qudit unitary operations into the sequence of native pulses. All of them provide Python packages that help execute and compare results on a $d \times d$ Python matrix of the type `np.ndarray`. We compare existing methods with our developed decompositions in Tables 2 and A1. All experiments were conducted using a MacBook Pro 2020 with an Apple M1 chip.

Table 2. Comparison of decompositions produced by the given methods in terms of transition count. **TO** means that the execution exceeded the 1 min timeout.

Method	X_d^{+1}			X_d^{-1}			H_d			U_d			**Two-Qubit Gate**					
	$d = 4$	$d = 5$	$d = 6$	$d = 4$	$d = 5$	$d = 6$	$d = 4$	$d = 5$	$d = 6$	$d = 4$	$d = 5$	$d = 6$	RXX	RZZ	CZ	CX	CH	SWAP
QSearchPass [87]	3	4	5	3	4	5	7	11	17	7	12	16	6	0	0	3	3	1
QSweepPass [85]	5	6	7	3	4	5	7	11	16	7	11	16	6	2	2	5	5	3
LocQRPass [88]	3	4	5	3	4	5	6	10	15	6	10	15	6	0	0	3	3	1
LocAdaPass [88]	3	4	5	3	4	5	6	**TO**	**TO**	7	**TO**	**TO**	4	0	0	2	2	1
TAQR	3	4	5	3	4	5	6	10	15	6	10	15	6	0	0	3	3	1
TAQR Adaptive	3	4	5	3	4	5	6	10	15	6	10	15	6	0	0	3	3	1
(**a**) Line transition graph.																		
QSearchPass [87]	3	4	5	3	4	5	6	12	16	7	11	16	4	0	0	3	3	3
LocQRPass [88]	7	10	13	7	10	13	16	28	43	16	28	43	16	0	0	9	9	3
LocAdaPass [88]	4	6	7	4	5	6	8	15	24	8	15	24	3	0	0	2	2	2
TAQR	5	6	9	3	4	5	5	10	15	6	10	15	4	0	0	3	3	3
TAQR Adaptive	3	4	5	3	4	5	6	10	15	6	10	15	4	0	0	3	3	3
(**b**) Star transition graph.																		
QSearchPass [87]	3	4	6	3	4	5	7	12	13	7	11	16	2	0	0	1	1	1
LocQRPass [88]	7	10	13	7	10	13	14	26	41	14	26	41	14	0	0	7	7	1
LocAdaPass [88]	4	6	7	4	5	6	9	14	21	9	14	21	2	0	0	1	1	1
TAQR	3	6	7	3	4	5	6	10	14	6	10	15	2	0	0	1	1	1
TAQR Adaptive	3	4	5	3	4	5	6	10	14	6	10	15	2	0	0	1	1	1
(**c**) Bipartite transition graph.																		

The first approach is a *BQSKIT* framework that provides a generic method for numerical synthesis of unitary operations. It supports a qudit unitary synthesis for multiqubit and multiqudit circuits. There exist several methods in *BQSKIT* that allow one to decompose qudit unitary operations. First is the *QSearchPass* [87] method, whose main purpose is to synthesize multiqubit circuits using the native gate set according to qubit connectivity, though it is not limited only to qubits processing and can be utilized to synthesize single-qudit unitary operations. We can obtain desired decomposition by declaring custom parametric qudit gates, which correspond to allowed transition R^{ij} and phase shift P^{k} operations. Notably, this method is not deterministic, so we run each experiment 20 times and take the median value for decomposition length. Listing A1 contains Python code to invoke *QSearchPass*. The second method is called *QSweepPass* [85], which is not part of *BQSKIT* but relies on its transpilation workflow. It uses numerical and analytical results to construct a qudit unitary operation in terms of two-level operations. However, it does not support arbitrary selection rules and only targets superconducting transition topology. Listing A2 contains Python code to invoke *QSweepPass*.

The second approach is provided by *MQT.Qudits* [88] framework for mixed-dimensional quantum computing, which is part of the *Munich Quantum Toolkit*. It is possible to pass a qudit transition graph with corresponding weights into the *LocQRPass* method and obtain the sequence of allowed gates as decomposition. This framework also supports the adaptive gate transpilation mode via the *LocAdaPass* method that can reduce the length of the decomposition. Listing A3 contains Python code to invoke *MQT.Qudits*.

In Table 2, our method is called *Transition-Aware QR decomposition*, or *TAQR* for simplicity, for static decomposition and *TAQR Adaptive* for adaptive decomposition. In our comparison, we are targeting three types of selection rules given as an undirected graph for different qudit platforms. The first considered platform has *the line transition graph* (Table 2a), where only transitions on neighboring levels are allowed. Superconducting qudits and photonic circuits have this type of selection rules. We can extend the line transition graph to any number of levels d. The second platform consists of trapped-ion qudits. Overall, this type of qudit can have any arbitrary transitions selected by atomic physics rules. Though, particular trapped-ion devices provide *the star transition graph* (Table 2b) or *the bipartite transition graph* (Table 2c) of the size d. *The star transition graph* is the special case of *the bipartite transition graph* with partition $p = 1$. *The bipartite transition graph* with a partition p is the graph with two disjoint subsets with sizes p and $d-p$, that is, every node in the first subset is connected via an edge to each node in the second subset. We use bipartite graph with $p = 2$ in our experiments.

Each method produces decomposition for each given platform for one of the following single-qudit operations:

- State number increment/decrement gate X_d^{+1} and X_d^{-1}, which have the form as in Equation (4).
- Quantum Fourier Transform with a size d that is denoted H_d in Equation (7).
- Uniformly distributed (using Haar measure procedure) unitary matrix U_d. Since the matrix is random, we run each decomposition on 100 generated matrices, then take median value for decomposition length.
- Two-qubit matrix $\in U(4)$ that represents the action on two qubits embedded into the qudit with $d = 4$. Chosen gates are as follows: $\mathsf{RXX}(\pi/2)$, $\mathsf{RZZ}(\pi/2)$, CZ, CX, CH, and SWAP.

As a result, *Transition-Aware QR* decomposition performs as good as any other qudit synthesis framework for superconducting or photonic platforms. For the majority of qudit unitary operations, it produces the decomposition with the same length in a reasonable time (refer Table A1 in the Appendix A for execution time comparison). Thus, it can become

a drop-in replacement for the qudit transpiler pass for single-qudit operations for this type of platform.

Next, our method provides the shortest decomposition for trapped-ion platforms (with any arbitrary selection rules). Our result outperforms the average numerical decomposition using *QSearch* method that produces random decomposition on each invocation and takes significantly greater execution time. It also outperforms *MQT.Qudits* compilation passes, which produce more transitions and level-swaps for the same unitary. For generic unitary, our method is guaranteed to produce $d(d-1)/2$ transitions, which is the theoretical upper bound (the number of lower non-diagonal elements in unitary).

Furthermore, lastly, we have shown the potential benefits of using either *static* or *adaptive* decomposition. Static decomposition is computed once for each platform, which reduces decomposition time during the transpilation process. On the other hand, adaptive decomposition considers the sparsity pattern of the given unitary matrix and produces a sequence with fewer transitions for several cases, e.g., for the X_d operation.

The total error of single-qudit execution grows with the number of transitions involved. The error of single-qudit transition ε_{ij} is quantified as an operator norm of a difference between the hardware implementation of R^{ij} and its theoretical expression in Equation (16). Given the sequence of transitions $\mathcal{D}$, we can obtain the estimation for the total error ε_{tot} of unitary execution:

$$\varepsilon_{\text{tot}} \leq \sum_{ij \in \mathcal{D}} \varepsilon_{ij}. \tag{31}$$

Hence, minimizing the number of transitions in decomposition leads to more accurate computations. Since our approach produces shorter sequences for many cases, it also performs better in mitigation of execution errors on qudit hardware. Moreover, the algorithm is able to choose transition with minimal error ε_{ij} on the particular steps, if we are able to provide the estimation for transition error beforehand.

Notably, while *LocAdaPass* produces shorter decomposition for two-qubit gates embedded into the ququart, it also permutes levels virtually. This fact makes sense for optimization of the whole qudit circuit, though we should permute rows virtually or use level-swap gates to obtain proper unitary. Potentially, our method can also benefit from virtual level-swaps when it is a part of the whole qudit circuit workflow.

7. Conclusions

In this paper, we have proposed a method that converts a d-level qudit unitary operation into the sequence of the allowed transitions between qudit levels. It considers arbitrary selection rules of the qudit system and produces the optimal decomposition sequence in terms of the number of transitions. Moreover, we show that the decomposition consists of no more than $d(d-1)/2$ transitions, which is the theoretical upper bound for an arbitrary unitary matrix $d \times d$.

Decomposition could be performed once for each qudit platform of interest before the transpilation, then applied to any unitary matrix to obtain parameters of transitions. This is the essence of the *static* decomposition. On the other hand, we have proposed the *adaptive* modification that utilizes the sparsity pattern of the matrix and the transition graph simultaneously. Such treatment could reduce the number of transitions even more for some operations, such as level permutation.

As we have shown, the crucial advantage of our approach is that it does not distinguish between qudit platforms and produces similar optimal decomposition for any transition graph. Most trapped-ion qudit platforms could benefit from utilizing this approach due to complex selection rules on operated levels.

Thus, the developed algorithm with the *static* and *adaptive* decompositions could become a drop-in replacement for single-qudit transpilation pass within qudit transpiler for multiqudit circuits. Multiqudit operations have an analytic decomposition in terms of native two-qudit operations and single-qudit generic unitary operation. TAQR could be utilized to decompose generic operations into an optimized sequence of native single-qudit operations and reduce single-qudit errors, therefore mitigating overall circuit execution error.

Author Contributions: Conceptualization, A.S.N.; Software, D.A.D.; Validation, E.O.K.; Investigation, D.A.D.; Writing—original draft, D.A.D.; Writing—review and editing, A.K.F., E.O.K. and A.S.N.; Supervision, A.S.N. and A.K.F.; Funding acquisition, A.S.N. and A.K.F. All authors have read and agreed to the published version of the manuscript.

Funding: The work of A.S.N. and D.A.D. was supported by RSF Grant No. 24-71-00084 (developing methods of quantum algorithms transpilation for the acceleration of their execution; Sections 4–6). The work of E.O.K. and A.K.F. was supported by the Priority 2030 program at the NUST "MISIS" under the project K1-2022-027 (exploring existing qudit gates; Sections 2 and 3).

Institutional Review Board Statement: Not applicable.

Data Availability Statement: The source code of the *TAQR* comparison with other methods is available at https://github.com/d-drozhzhin/qudit-unitary-decomposition (accessed on 30 December 2025). The source code of the developed method is available on reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Comparison with Existing Decomposition Methods in Terms of Execution Time

Table A1. Comparison of decompositions produced by given methods in terms of execution time (in milliseconds). **TO** means that the execution exceeded the 1 min timeout.

Method	X_d^{+1}			X_d^{-1}			H_d			U_d			**Two-Qubit Gate**					
	$d = 4$	$d = 5$	$d = 6$	$d = 4$	$d = 5$	$d = 6$	$d = 4$	$d = 5$	$d = 6$	$d = 4$	$d = 5$	$d = 6$	RXX	RZZ	CZ	CX	CH	SWAP
QSearchPass [87]	3182	14,078	10,513	3160	7046	14,449	3712	6264	16,764	3993	7324	16,364	10,378	2554	2204	2891	3170	2550
QSweepPass [85]	61.95	96.84	157.01	13.02	17.37	24.71	145.45	246.6	427.24	155.2	323.21	599.99	57.8	48.01	39.79	52.21	74.06	44.62
LocQRPass [88]	0.227	0.294	0.328	0.239	0.298	0.552	0.421	0.785	0.789	0.396	0.553	0.998	0.361	0.097	0.083	0.229	0.237	0.147
LocAdaPass [88]	0.706	0.93	1.223	1.263	3.23	8.81	1427.2	**TO**	**TO**	1089.1	**TO**	**TO**	0.929	0.179	0.15	0.472	0.478	0.249
TAQR	0.28	0.347	0.487	0.281	0.425	0.468	0.435	0.688	1.009	0.494	0.692	1.035	0.407	0.117	0.105	0.265	0.282	0.166
TAQR Adaptive	1.261	2.318	4.456	1.253	2.341	4.146	1.553	2.854	4.878	1.67	2.857	4.861	1.502	0.597	0.594	1.638	1.185	0.859
(**a**) Line transition graph.																		
QSearchPass [87]	2821	3067	3742	2929	3057	4241	4147	7187	17,974	3949	7785	17,719	3461	2602	2200	3068	3107	3051
LocQRPass [88]	0.392	0.896	0.625	0.361	0.514	0.654	0.842	1.361	2.036	1.186	1.173	1.947	0.713	0.101	0.083	0.407	0.408	0.208
LocAdaPass [88]	1.052	2.222	2.545	1.005	1.509	2.144	4.606	14.07	36.08	4.181	13.41	39.36	1.254	0.168	0.143	0.769	0.711	0.396
TAQR	0.663	0.472	0.623	0.274	0.367	0.459	0.397	1.09	0.99	0.431	0.735	1.048	0.314	0.113	0.104	0.254	0.279	0.268
TAQR Adaptive	1.359	2.204	2.374	1.059	1.631	2.355	1.379	2.335	3.578	1.38	2.33	3.581	1.128	0.398	0.353	1.118	1.009	1.249
(**b**) Star transition graph.																		
QSearchPass [87]	2446	2835	4177	2524	2949	4469	3159	7087	11,866	3525	7992	19,952	2276	2289	2267	2239	2229	2268
LocQRPass [88]	0.352	0.489	0.651	0.371	0.553	0.775	0.761	1.27	1.823	0.658	1.152	1.933	0.619	0.099	0.086	0.354	0.369	0.15
LocAdaPass [88]	0.872	1.65	2.437	0.953	1.565	2.339	3.658	13.0	37.24	826.7	12.02	49.2	1.102	0.171	0.138	0.625	0.606	0.254
TAQR	0.286	0.439	0.576	0.262	0.387	0.47	0.441	0.736	0.966	0.438	0.677	1.032	0.21	0.116	0.108	0.161	0.169	0.163
TAQR Adaptive	1.203	1.952	3.11	1.213	1.821	2.542	1.57	2.568	3.822	1.574	2.594	3.909	1.027	0.418	0.405	0.93	0.83	0.987
(**c**) Bipartite transition graph.																		

Appendix B. Python Code to Execute Decomposition

We use the following Python code to perform a comparison of different methods for qudit unitary decomposition. Each method produces a qudit circuit format that contains only allowed transition gates and phase-shift gates.

Listing A1. Invoke *QSearchPass* decomposition using `bqskit` package with the version `1.2.1`.

```python
import numpy as np
import networkx as nx

from bqskit.ir import Circuit
from bqskit.ir.gates import EmbeddedGate, U1Gate, U1qGate
from bqskit.ir.opt.cost import HilbertSchmidtCostGenerator
from bqskit.ir.opt.minimizers import LBFGSMinimizer
from bqskit.ir.opt.instantiaters import Minimization
from bqskit.qis import UnitaryMatrix
from bqskit.compiler import MachineModel, Compiler, Workflow
from bqskit.passes import SetTargetPass, SetModelPass
from bqskit.passes import QSearchSynthesisPass, SingleQuditLayerGenerator
from bqskit.passes import ScanningGateRemovalPass


class QSearch:
    def decompose(self, utry: np.ndarray, trans: nx.Graph) -> Circuit:
        assert len(utry.shape) == 2 and utry.shape[0] == utry.shape[1]
        d = utry.shape[0]

        target = UnitaryMatrix(utry, [d])

        model = MachineModel(
            1,
            None,
            [EmbeddedGate(U1Gate(), [d], [0, k]) for k in range(1, d)]
            + [EmbeddedGate(U1qGate(), [d], [i, j]) for i, j in trans.edges],
            [d],
        )

        workflow = Workflow(
            [
                SetTargetPass(target),
                SetModelPass(model),
                QSearchSynthesisPass(
                    max_layer=d**2,
                    layer_generator=SingleQuditLayerGenerator(
                        None, allow_repeats=False
                    ),
                    instantiate_options={
                        "method": Minimization(
                            cost_fn_gen=HilbertSchmidtCostGenerator(),
                            minimizer=LBFGSMinimizer(),
                        )
                    },
                ),
                ScanningGateRemovalPass(
                    instantiate_options={
                        "multistarts": 2,
                        "max_iters": 1000,
                        "min_iters": 10,
                    }
                ),
            ]
        )

        with Compiler(num_workers=8, runtime_log_level=40) as compiler:
            circuit = compiler.compile(Circuit(1, [d]), workflow=workflow)
        return circuit
```

Listing A2. Invoke *QSweepPass* decomposition using `qsweep` package from https://github.com/edyounis/QSweep.git (accessed on 30 December 2025) repository at the commit `b621547`.

```
import numpy as np

from bqskit.ir import Circuit
from bqskit.ir.gates import U1Gate, U1qGate
from bqskit.qis import UnitaryMatrix
from qsweep import QSweepPass


class QSweep:
    def decompose(self, utry: np.ndarray) -> Circuit:
        assert len(utry.shape) == 2 and utry.shape[0] == utry.shape[1]
        d = utry.shape[0]

        target = UnitaryMatrix(utry, [d])
        qsweep_pass = QSweepPass({U1Gate(), U1qGate()})
        return qsweep_pass.synthesize_non_async(target)
```

Listing A3. Invoke *MQT.Qudits* decomposition using `mqt.qudits` package with the version `0.4.0`.

```
import numpy as np
import networkx as nx

from mqt.qudits.core import LevelGraph
from mqt.qudits.compiler import QuditCompiler
from mqt.qudits.quantum_circuit import QuantumCircuit
from mqt.qudits.simulation import MQTQuditProvider
from mqt.qudits.simulation.backends.backendv2 import Backend


class FakeBackend(Backend):
    def __init__(self, trans: nx.Graph, dim: int):
        super().__init__(provider=MQTQuditProvider(), name="FakeBackend")

        graph = LevelGraph(
            edges=trans.edges,
            nodes=list(range(dim)),
            nodes_physical_mapping=list(range(dim)),
            initialization_nodes=[0],
        )
        self._energy_level_graphs: list[LevelGraph] = [graph]

    @property
    def energy_level_graphs(self) -> list[LevelGraph]:
        return self._energy_level_graphs

    def run(self, circuit, **options):
        pass


class MqtQudits:
    def __init__(self, method: str):
        assert method == "LocQRPass" or method == "LocAdaPass"
        self.method = method

    def decompose(self, utry: np.ndarray, trans: nx.Graph) -> QuantumCircuit:
        assert len(utry.shape) == 2 and utry.shape[0] == utry.shape[1]
        d = utry.shape[0]

        circuit = QuantumCircuit(1, [d])
        circuit.cu_one(0, utry)

        backend = FakeBackend(trans, d)
        return QuditCompiler().compile(backend, circuit, [self.method])
```

References

1. Brassard, G.; Chuang, I.; Lloyd, S.; Monroe, C. Quantum computing. *Proc. Natl. Acad. Sci. USA* **1998**, *95*, 11032–11033. [CrossRef]
2. Ladd, T.D.; Jelezko, F.; Laflamme, R.; Nakamura, Y.; Monroe, C.; O'Brien, J.L. Quantum computers. *Nature* **2010**, *464*, 45–53. [CrossRef] [PubMed]
3. Fedorov, A.K.; Gisin, N.; Beloussov, S.M.; Lvovsky, A.I. Quantum computing at the quantum advantage threshold: A down-to-business review. *arXiv* **2022**, arXiv:2203.17181. [CrossRef]
4. Lloyd, S. Universal Quantum Simulators. *Science* **1996**, *273*, 1073–1078. [CrossRef]
5. Farhi, E.; Goldstone, J.; Gutmann, S. A Quantum Approximate Optimization Algorithm. *arXiv* **2014**, arXiv:1411.4028. [CrossRef]

6. Shor, P. Algorithms for quantum computation: Discrete logarithms and factoring. In Proceedings of the Proceedings 35th Annual Symposium on Foundations of Computer Science, Santa Fe, NM, USA, 20–22 November 1994; pp. 124–134. [CrossRef]
7. Cerezo, M.; Arrasmith, A.; Babbush, R.; Benjamin, S.C.; Endo, S.; Fujii, K.; McClean, J.R.; Mitarai, K.; Yuan, X.; Cincio, L.; et al. Variational quantum algorithms. *Nat. Rev. Phys.* **2021**, *3*, 625–644. [CrossRef]
8. Bharti, K.; Cervera-Lierta, A.; Kyaw, T.H.; Haug, T.; Alperin-Lea, S.; Anand, A.; Degroote, M.; Heimonen, H.; Kottmann, J.S.; Menke, T.; et al. Noisy intermediate-scale quantum (NISQ) algorithms. *Rev. Mod. Phys.* **2022**, *94*, 015004. [CrossRef]
9. Ebadi, S.; Wang, T.T.; Levine, H.; Keesling, A.; Semeghini, G.; Omran, A.; Bluvstein, D.; Samajdar, R.; Pichler, H.; Ho, W.W.; et al. Quantum phases of matter on a 256-atom programmable quantum simulator. *Nature* **2021**, *595*, 227–232. [CrossRef]
10. Arute, F.; Arya, K.; Babbush, R.; Bacon, D.; Bardin, J.C.; Barends, R.; Biswas, R.; Boixo, S.; Brandao, F.G.S.L.; Buell, D.A.; et al. Quantum supremacy using a programmable superconducting processor. *Nature* **2019**, *574*, 505–510. [CrossRef] [PubMed]
11. Wu, Y.; Bao, W.S.; Cao, S.; Chen, F.; Chen, M.C.; Chen, X.; Chung, T.H.; Deng, H.; Du, Y.; Fan, D.; et al. Strong Quantum Computational Advantage Using a Superconducting Quantum Processor. *Phys. Rev. Lett.* **2021**, *127*, 180501. [CrossRef]
12. Xue, X.; Russ, M.; Samkharadze, N.; Undseth, B.; Sammak, A.; Scappucci, G.; Vandersypen, L.M.K. Quantum logic with spin qubits crossing the surface code threshold. *Nature* **2022**, *601*, 343–347. [CrossRef] [PubMed]
13. Madzik, M.T.; Asaad, S.; Youssry, A.; Joecker, B.; Rudinger, K.M.; Nielsen, E.; Young, K.C.; Proctor, T.J.; Baczewski, A.D.; Laucht, A.; et al. Precision tomography of a three-qubit donor quantum processor in silicon. *Nature* **2022**, *601*, 348–353. [CrossRef]
14. Noiri, A.; Takeda, K.; Nakajima, T.; Kobayashi, T.; Sammak, A.; Scappucci, G.; Tarucha, S. Fast universal quantum gate above the fault-tolerance threshold in silicon. *Nature* **2022**, *601*, 338–342. [CrossRef]
15. Zhong, H.S.; Wang, H.; Deng, Y.H.; Chen, M.C.; Peng, L.C.; Luo, Y.H.; Qin, J.; Wu, D.; Ding, X.; Hu, Y.; et al. Quantum computational advantage using photons. *Science* **2020**, *370*, 1460–1463. [CrossRef]
16. Madsen, L.S.; Laudenbach, F.; Askarani, M.F.; Rortais, F.; Vincent, T.; Bulmer, J.F.F.; Miatto, F.M.; Neuhaus, L.; Helt, L.G.; Collins, M.J.; et al. Quantum computational advantage with a programmable photonic processor. *Nature* **2022**, *606*, 75–81. [CrossRef] [PubMed]
17. Scholl, P.; Schuler, M.; Williams, H.J.; Eberharter, A.A.; Barredo, D.; Schymik, K.N.; Lienhard, V.; Henry, L.P.; Lang, T.C.; Lahaye, T.; et al. Quantum simulation of 2D antiferromagnets with hundreds of Rydberg atoms. *Nature* **2021**, *595*, 233–238. [CrossRef] [PubMed]
18. Henriet, L.; Beguin, L.; Signoles, A.; Lahaye, T.; Browaeys, A.; Reymond, G.O.; Jurczak, C. Quantum computing with neutral atoms. *Quantum* **2020**, *4*, 327. [CrossRef]
19. Graham, T.M.; Song, Y.; Scott, J.; Poole, C.; Phuttitarn, L.; Jooya, K.; Eichler, P.; Jiang, X.; Marra, A.; Grinkemeyer, B.; et al. Multi-qubit entanglement and algorithms on a neutral-atom quantum computer. *Nature* **2022**, *604*, 457–462. [CrossRef]
20. Zhang, J.; Pagano, G.; Hess, P.W.; Kyprianidis, A.; Becker, P.; Kaplan, H.; Gorshkov, A.V.; Gong, Z.X.; Monroe, C. Observation of a many-body dynamical phase transition with a 53-qubit quantum simulator. *Nature* **2017**, *551*, 601–604. [CrossRef]
21. Blatt, R.; Roos, C.F. Quantum simulations with trapped ions. *Nat. Phys.* **2012**, *8*, 277–284. [CrossRef]
22. Hempel, C.; Maier, C.; Romero, J.; McClean, J.; Monz, T.; Shen, H.; Jurcevic, P.; Lanyon, B.P.; Love, P.; Babbush, R.; et al. Quantum Chemistry Calculations on a Trapped-Ion Quantum Simulator. *Phys. Rev. X* **2018**, *8*, 031022. [CrossRef]
23. Kiktenko, E.O.; Nikolaeva, A.S.; Fedorov, A.K. Colloquium: Qudits for decomposing multiqubit gates and realizing quantum algorithms. *Rev. Mod. Phys.* **2025**, *97*, 021003. [CrossRef]
24. Farhi, E.; Gutmann, S. Analog analogue of a digital quantum computation. *Phys. Rev. A* **1998**, *57*, 2403–2406. [CrossRef]
25. Kessel', A.R.; Ermakov, V.L. Multiqubit spin. *J. Exp. Theor. Phys. Lett.* **1999**, *70*, 61–65. [CrossRef]
26. Kessel', A.R.; Ermakov, V.L. Physical implementation of three-qubit gates on a separate quantum particle. *J. Exp. Theor. Phys. Lett.* **2000**, *71*, 307–309. [CrossRef]
27. Kessel, A.R.; Yakovleva, N.M. Implementation schemes in NMR of quantum processors and the Deutsch-Jozsa algorithm by using virtual spin representation. *Phys. Rev. A* **2002**, *66*, 062322. [CrossRef]
28. Muthukrishnan, A.; Stroud, C.R. Multivalued logic gates for quantum computation. *Phys. Rev. A* **2000**, *62*, 052309. [CrossRef]
29. Nielsen, M.A.; Bremner, M.J.; Dodd, J.L.; Childs, A.M.; Dawson, C.M. Universal simulation of Hamiltonian dynamics for quantum systems with finite-dimensional state spaces. *Phys. Rev. A* **2002**, *66*, 022317. [CrossRef]
30. Wang, X.; Sanders, B.C.; Berry, D.W. Entangling power and operator entanglement in qudit systems. *Phys. Rev. A* **2003**, *67*, 042323. [CrossRef]
31. Klimov, A.B.; Guzmán, R.; Retamal, J.C.; Saavedra, C. Qutrit quantum computer with trapped ions. *Phys. Rev. A* **2003**, *67*, 062313. [CrossRef]
32. Bagan, E.; Baig, M.; Muñoz Tapia, R. Minimal measurements of the gate fidelity of a qudit map. *Phys. Rev. A* **2003**, *67*, 014303. [CrossRef]

33. Vlasov, A.Y. Algebra of quantum computations with higher dimensional systems. In *Proceedings of the First International Symposium on Quantum Informatics*; Ozhigov, Y.I., Ed.; International Society for Optics and Photonics, SPIE: Bellingham, DC, USA, 2003; Volume 5128, pp. 29–36. [CrossRef]
34. Greentree, A.D.; Schirmer, S.G.; Green, F.; Hollenberg, L.C.L.; Hamilton, A.R.; Clark, R.G. Maximizing the Hilbert Space for a Finite Number of Distinguishable Quantum States. *Phys. Rev. Lett.* **2004**, *92*, 097901. [CrossRef]
35. O'Leary, D.P.; Brennen, G.K.; Bullock, S.S. Parallelism for quantum computation with qudits. *Phys. Rev. A* **2006**, *74*, 032334. [CrossRef]
36. Ralph, T.C.; Resch, K.J.; Gilchrist, A. Efficient Toffoli gates using qudits. *Phys. Rev. A* **2007**, *75*, 022313. [CrossRef]
37. Lanyon, B.P.; Weinhold, T.J.; Langford, N.K.; O'Brien, J.L.; Resch, K.J.; Gilchrist, A.; White, A.G. Manipulating Biphotonic Qutrits. *Phys. Rev. Lett.* **2008**, *100*, 060504. [CrossRef] [PubMed]
38. Ionicioiu, R.; Spiller, T.P.; Munro, W.J. Generalized Toffoli gates using qudit catalysis. *Phys. Rev. A* **2009**, *80*, 012312. [CrossRef]
39. Neeley, M.; Ansmann, M.; Bialczak, R.C.; Hofheinz, M.; Lucero, E.; O'Connell, A.D.; Sank, D.; Wang, H.; Wenner, J.; Cleland, A.N.; et al. Emulation of a Quantum Spin with a Superconducting Phase Qudit. *Science* **2009**, *325*, 722–725. [CrossRef]
40. Lanyon, B.P.; Barbieri, M.; Almeida, M.P.; Jennewein, T.; Ralph, T.C.; Resch, K.J.; Pryde, G.J.; O'Brien, J.L.; Gilchrist, A.; White, A.G. Simplifying quantum logic using higher-dimensional Hilbert spaces. *Nat. Phys.* **2009**, *5*, 134–140. [CrossRef]
41. Ivanov, S.S.; Tonchev, H.S.; Vitanov, N.V. Time-efficient implementation of quantum search with qudits. *Phys. Rev. A* **2012**, *85*, 062321. [CrossRef]
42. Mischuck, B.E.; Merkel, S.T.; Deutsch, I.H. Control of inhomogeneous atomic ensembles of hyperfine qudits. *Phys. Rev. A* **2012**, *85*, 022302. [CrossRef]
43. Fedorov, A.; Steffen, L.; Baur, M.; da Silva, M.P.; Wallraff, A. Implementation of a Toffoli gate with superconducting circuits. *Nature* **2012**, *481*, 170–172. [CrossRef]
44. Li, B.; Yu, Z.H.; Fei, S.M. Geometry of Quantum Computation with Qutrits. *Sci. Rep.* **2013**, *3*, 2594. [CrossRef]
45. Svetitsky, E.; Suchowski, H.; Resh, R.; Shalibo, Y.; Martinis, J.M.; Katz, N. Hidden two-qubit dynamics of a four-level Josephson circuit. *Nat. Commun.* **2014**, *5*, 5617. [CrossRef] [PubMed]
46. Peterer, M.J.; Bader, S.J.; Jin, X.; Yan, F.; Kamal, A.; Gudmundsen, T.J.; Leek, P.J.; Orlando, T.P.; Oliver, W.D.; Gustavsson, S. Coherence and Decay of Higher Energy Levels of a Superconducting Transmon Qubit. *Phys. Rev. Lett.* **2015**, *114*, 010501. [CrossRef]
47. Kiktenko, E.O.; Fedorov, A.K.; Man'ko, O.V.; Man'ko, V.I. Multilevel superconducting circuits as two-qubit systems: Operations, state preparation, and entropic inequalities. *Phys. Rev. A* **2015**, *91*, 042312. [CrossRef]
48. Kiktenko, E.; Fedorov, A.; Strakhov, A.; Man'ko, V. Single qudit realization of the Deutsch algorithm using superconducting many-level quantum circuits. *Phys. Lett. A* **2015**, *379*, 1409–1413. [CrossRef]
49. Braumüller, J.; Cramer, J.; Schlör, S.; Rotzinger, H.; Radtke, L.; Lukashenko, A.; Yang, P.; Skacel, S.T.; Probst, S.; Marthaler, M.; et al. Multiphoton dressing of an anharmonic superconducting many-level quantum circuit. *Phys. Rev. B* **2015**, *91*, 054523. [CrossRef]
50. Song, C.; Su, S.L.; Wu, J.L.; Wang, D.Y.; Ji, X.; Zhang, S. Generation of tree-type three-dimensional entangled states via adiabatic passage. *Phys. Rev. A* **2016**, *93*, 062321. [CrossRef]
51. Frydryszak, A.; Jakóbczyk, L.; Ługiewicz, P. Determining quantum correlations in bipartite systems - from qubit to qutrit and beyond. *J. Phys. Conf. Ser.* **2017**, *804*, 012016. [CrossRef]
52. Godfrin, C.; Ferhat, A.; Ballou, R.; Klyatskaya, S.; Ruben, M.; Wernsdorfer, W.; Balestro, F. Operating Quantum States in Single Magnetic Molecules: Implementation of Grover's Quantum Algorithm. *Phys. Rev. Lett.* **2017**, *119*, 187702. [CrossRef]
53. Bocharov, A.; Roetteler, M.; Svore, K.M. Factoring with qutrits: Shor's algorithm on ternary and metaplectic quantum architectures. *Phys. Rev. A* **2017**, *96*, 012306. [CrossRef]
54. Kues, M.; Reimer, C.; Roztocki, P.; Cortés, L.R.; Sciara, S.; Wetzel, B.; Zhang, Y.; Cino, A.; Chu, S.T.; Little, B.E.; et al. On-chip generation of high-dimensional entangled quantum states and their coherent control. *Nature* **2017**, *546*, 622–626. [CrossRef] [PubMed]
55. Gokhale, P.; Baker, J.M.; Duckering, C.; Brown, N.C.; Brown, K.R.; Chong, F.T. Asymptotic Improvements to Quantum Circuits via Qutrits. In Proceedings of the 46th International Symposium on Computer Architecture, New York, NY, USA, 22–26 June 2019; ISCA '19, pp. 554–566. [CrossRef]
56. Luo, Y.H.; Zhong, H.S.; Erhard, M.; Wang, X.L.; Peng, L.C.; Krenn, M.; Jiang, X.; Li, L.; Liu, N.L.; Lu, C.Y.; et al. Quantum Teleportation in High Dimensions. *Phys. Rev. Lett.* **2019**, *123*, 070505. [CrossRef]
57. Low, P.J.; White, B.M.; Cox, A.A.; Day, M.L.; Senko, C. Practical trapped-ion protocols for universal qudit-based quantum computing. *Phys. Rev. Res.* **2020**, *2*, 033128. [CrossRef]
58. Jin, Z.; Gong, W.J.; Zhu, A.D.; Zhang, S.; Qi, Y.; Su, S.L. Dissipative preparation of qutrit entanglement via periodically modulated Rydberg double antiblockade. *Opt. Express* **2021**, *29*, 10117–10133. [CrossRef]
59. Sawant, R.; Blackmore, J.A.; Gregory, P.D.; Mur-Petit, J.; Jaksch, D.; Aldegunde, J.; Hutson, J.M.; Tarbutt, M.R.; Cornish, S.L. Ultracold polar molecules as qudits. *New J. Phys.* **2020**, *22*, 013027. [CrossRef]

60. Pavlidis, A.; Floratos, E. Quantum-Fourier-transform-based quantum arithmetic with qudits. *Phys. Rev. A* **2021**, *103*, 032417. [CrossRef]
61. Rambow, P.; Tian, M. Reduction of circuit depth by mapping qubit-based quantum gates to a qudit basis. *arXiv* **2021**, arXiv:2109.09902. [CrossRef]
62. Nikolaeva, A.S.; Kiktenko, E.O.; Fedorov, A.K. Efficient realization of quantum algorithms with qudits. *EPJ Quantum Technol.* **2024**, *11*, 1–25. [CrossRef]
63. Nikolaeva, A.S.; Kiktenko, E.O.; Fedorov, A.K. Universal quantum computing with qubits embedded in trapped-ion qudits. *Phys. Rev. A* **2024**, *109*, 022615. [CrossRef]
64. Drozhzhin, D.A.; Nikolaeva, A.S.; Kiktenko, E.O.; Fedorov, A.K. Transpiling Quantum Assembly Language Circuits to a Qudit Form. *Entropy* **2024**, *26*, 1129. [CrossRef]
65. Barenco, A.; Bennett, C.H.; Cleve, R.; DiVincenzo, D.P.; Margolus, N.; Shor, P.; Sleator, T.; Smolin, J.A.; Weinfurter, H. Elementary gates for quantum computation. *Phys. Rev. A* **1995**, *52*, 3457–3467. [CrossRef]
66. Liu, W.Q.; Wei, H.R.; Kwek, L.C. Low-Cost Fredkin Gate with Auxiliary Space. *Phys. Rev. Appl.* **2020**, *14*, 054057. [CrossRef]
67. Baker, J.M.; Duckering, C.; Chong, F.T. Efficient Quantum Circuit Decompositions via Intermediate Qudits. In Proceedings of the 2020 IEEE 50th International Symposium on Multiple-Valued Logic (ISMVL), Miyazaki, Japan, 9–11 November 2020; pp. 303–308. [CrossRef]
68. Kiktenko, E.O.; Nikolaeva, A.S.; Xu, P.; Shlyapnikov, G.V.; Fedorov, A.K. Scalable quantum computing with qudits on a graph. *Phys. Rev. A* **2020**, *101*, 022304. [CrossRef]
69. Liu, W.Q.; Wei, H.R.; Kwek, L.C. Universal Quantum Multi-Qubit Entangling Gates with Auxiliary Spaces. *Adv. Quantum Technol.* **2022**, *5*, 2100136. [CrossRef]
70. Galda, A.; Cubeddu, M.; Kanazawa, N.; Narang, P.; Earnest-Noble, N. Implementing a Ternary Decomposition of the Toffoli Gate on Fixed-Frequency Transmon Qutrits. *arXiv* **2021**, arXiv:2109.00558.
71. Gu, X.; Allcock, J.; An, S.; Liu, Y.x. Efficient multi-qubit subspace rotations via topological quantum walks. *arXiv* **2021**, arXiv:2111.06534. [CrossRef]
72. Hill, A.D.; Hodson, M.J.; Didier, N.; Reagor, M.J. Realization of arbitrary doubly-controlled quantum phase gates. *arXiv* **2021**, arXiv:2108.01652. [CrossRef]
73. Chi, Y.; Huang, J.; Zhang, Z.; Mao, J.; Zhou, Z.; Chen, X.; Zhai, C.; Bao, J.; Dai, T.; Yuan, H.; et al. A programmable qudit-based quantum processor. *Nat. Commun.* **2022**, *13*, 1166. [CrossRef] [PubMed]
74. Ringbauer, M.; Meth, M.; Postler, L.; Stricker, R.; Blatt, R.; Schindler, P.; Monz, T. A universal qudit quantum processor with trapped ions. *Nat. Phys.* **2022**, *18*, 1053–1057. [CrossRef]
75. Aksenov, M.A.; Zalivako, I.V.; Semerikov, I.A.; Borisenko, A.S.; Semenin, N.V.; Sidorov, P.L.; Fedorov, A.K.; Khabarova, K.Y.; Kolachevsky, N.N. Realizing quantum gates with optically addressable $^{171}Yb^{+}$ ion qudits. *Phys. Rev. A* **2023**, *107*, 052612. [CrossRef]
76. Champion, E.; Wang, Z.; Parker, R.W.; Blok, M.S. Efficient Control of a Transmon Qudit Using Effective Spin-7/2 Rotations. *Phys. Rev. X* **2025**, *15*, 021096. [CrossRef]
77. Wang, Z.; Parker, R.W.; Champion, E.; Blok, M.S. High-E_J/E_C transmon qudits with up to 12 levels. *Phys. Rev. Appl.* **2025**, *23*, 034046. [CrossRef]
78. Yu, Y.; Chi, Y.; Zhai, C.; Huang, J.; Gong, Q.; Wang, J. Simulating Hamiltonian dynamics in a programmable photonic quantum processor using linear combinations of unitary operations. *arXiv* **2022**, arXiv:2211.06723.
79. Nikolaeva, A.S.; Kiktenko, E.O.; Fedorov, A.K. Decomposing the generalized Toffoli gate with qutrits. *Phys. Rev. A* **2022**, *105*, 032621. [CrossRef]
80. Nikolaeva, A.S.; Zalivako, I.V.; Borisenko, A.S.; Semenin, N.V.; Galstyan, K.P.; Korolkov, A.E.; Kiktenko, E.O.; Khabarova, K.Y.; Semerikov, I.A.; Fedorov, A.K.; et al. Scalable improvement of the generalized Toffoli gate realization using trapped-ion-based qutrits. *Phys. Rev. Lett.* **2024**, *135*, 060601. [CrossRef]
81. Brennen, G.; O'Leary, D.; Bullock, S. Criteria for exact qudit universality. *Phys. Rev. A* **2005**, *71*, 052318. [CrossRef]
82. Low, P.J.; White, B.; Senko, C. Control and readout of a 13-level trapped ion qudit. *npj Quantum Inf.* **2025**, *11*, 85. [CrossRef]
83. Low, P.J.; Zutt, N.C.F.; Tathed, G.A.; Senko, C. Quantum logic operations and algorithms in a single 25-level atomic qudit. *arXiv* **2025**, arXiv:2507.15799. [CrossRef]
84. Zalivako, I.V.; Nikolaeva, A.S.; Borisenko, A.S.; Korolkov, A.E.; Sidorov, P.L.; Galstyan, K.P.; Semenin, N.V.; Smirnov, V.N.; Aksenov, M.A.; Makushin, K.M.; et al. Towards multiqudit quantum processor based on a $^{171}Yb^{+}$ ion string: Realizing basic quantum algorithms. *Quantum Rep.* **2025**, *7*, 19. [CrossRef]
85. Younis, E.; Goss, N. QSweep: Pulse-Optimal Single-Qudit Synthesis. *arXiv* **2023**, arXiv:2312.09990.
86. Clements, W.R.; Humphreys, P.C.; Metcalf, B.J.; Kolthammer, W.S.; Walmsley, I.A. Optimal design for universal multiport interferometers. *Optica* **2016**, *3*, 1460–1465. [CrossRef]

87. Davis, M.G.; Smith, E.; Tudor, A.; Sen, K.; Siddiqi, I.; Iancu, C. Towards Optimal Topology Aware Quantum Circuit Synthesis. In Proceedings of the 2020 IEEE International Conference on Quantum Computing and Engineering (QCE), Denver, CO, USA, 12–16 October 2020; pp. 223–234. [CrossRef]
88. Mato, K.; Ringbauer, M.; Burgholzer, L.; Wille, R. MQT Qudits: A Software Framework for Mixed-Dimensional Quantum Computing. *arXiv* **2024**, arXiv:2410.02854. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.